FleetLogix — Documentación integral del proyecto de datos

Este documento reúne en un solo flujo las cuatro partes del proyecto FleetLogix: la generación de datos sintéticos, la optimización de consultas en PostgreSQL, la arquitectura AWS y la construcción del Data Warehouse con su proceso ETL.

Contenido

1. Descripción del proyecto
2. Generación de datos sintéticos
3. Optimización de consultas en PostgreSQL
4. Arquitectura AWS
5. Construcción del Data Warehouse y proceso ETL

1. Descripción del proyecto

FleetLogix es una empresa de transporte y logística que opera una flota de 200 vehículos realizando entregas de última milla en 5 ciudades principales. La empresa ha estado operando con sistemas legacy y hojas de cálculo, pero necesita modernizar su infraestructura de datos para competir en el mercado actual.

El objetivo es potenciar esta infraestructura: poblar todas las tablas con datos sintéticos masivos que respeten las relaciones del negocio, desarrollar queries para resolver problemas operativos reales, y diseñar una arquitectura cloud que permite análisis en tiempo real y toma de decisiones basada en datos.

Como punto de partida, se proporcionó un código base para la generación de datos sintéticos que presentaba funcionalidades incompletas y diversos errores que afectaban la lógica y la consistencia de la información generada. Por esta razón, fue necesario realizar un proceso de análisis, depuración y desarrollo para completar las secciones faltantes, corregir las inconsistencias detectadas y garantizar que los datos generados cumplieran con las relaciones, restricciones y volúmenes requeridos para el proyecto.

Objetivos

El proyecto busca cumplir los siguientes objetivos:

- Analizar el modelo relacional existente.
- Comprender las relaciones entre entidades.
- Generar más de 505.000 registros sintéticos.
- Mantener la integridad referencial.
- Respetar reglas de negocio.
- Validar la calidad de los datos generados.
- Implementar consultas de análisis empresarial.

Tecnologías utilizadas

Durante el desarrollo del proyecto se utilizaron las siguientes herramientas:

- PostgreSQL 15
- Python 3
- Faker
- NumPy
- Psycopg2
- Pandas
- DBeaver

2. Generación de datos sintéticos

2.1 Modelo de datos

El sistema está compuesto por seis tablas principales:

- Vehicles
- Drivers
- Routes
- Trips
- Deliveries
- Maintenance

Estas entidades representan el funcionamiento de una empresa dedicada al transporte y distribución de mercancías entre diferentes ciudades del país.

> **[Imagen del PDF original: diagrama entidad-relación (DBeaver) con las tablas `vehicles`, `drivers`, `routes`, `trips`, `deliveries` y `maintenance`.** - `vehicles`: vehicle_id, license_plate, vehicle_type, capacity_kg, fuel_type, acquisition_date, status. - `drivers`: driver_id, employee_code, first_name, last_name, license_number, license_expiry, phone, hire_date, status. - `routes`: route_id, route_code, origin_city, destination_city, distance_km, estimated_duration_hours, toll_cost. - `trips`: trip_id, vehicle_id, driver_id, route_id, departure_datetime, arrival_datetime, fuel_consumed_liters, total_weight_kg, status. - `deliveries`: delivery_id, trip_id, tracking_number, customer_name, delivery_address, package_weight_kg, scheduled_datetime, delivered_datetime, delivery_status, recipient_signature. - `maintenance`: maintenance_id, vehicle_id, maintenance_date, maintenance_type, description, cost, next_maintenance_date, performed_by.
>
> `trips` se relaciona con `vehicles`, `drivers` y `routes`; `deliveries` se relaciona con `trips`; `maintenance` se relaciona con `vehicles`.**

2.2 Proceso de generación

Para poblar la base de datos se desarrolló un script en Python completamente automatizado.

La generación se realizó respetando todas las relaciones entre tablas, evitando inconsistencias y simulando un comportamiento cercano a una operación logística real.

2.3 Registros generados

Tabla	Registros
Vehicles	200
Drivers	400
Routes	50
Trips	100000
Deliveries	400000
Maintenance	5000

Total: 505.650 registros

2.4 Reglas de negocio implementadas

Durante la generación de datos se respetaron las siguientes reglas:

Vehículos

- Distribución entre camiones grandes, camiones medianos, vans y motocicletas.
- Capacidades diferentes según el tipo de vehículo.
- Fechas de adquisición realistas.
- Vehículos activos y en mantenimiento.

Conductores

Cada conductor posee:

- Código de empleado único.
- Licencia válida.
- Tipo de licencia.
- Fecha de contratación.
- Estado activo o inactivo.

Rutas

Las rutas conectan las principales ciudades de Colombia:

- Bogotá
- Medellín
- Cali
- Barranquilla
- Cartagena

Cada ruta contiene:

- Distancia aproximada.
- Tiempo estimado.
- Valor de peajes.

Viajes

Cada viaje:

- Posee un vehículo existente,
- Posee un conductor existente,
- Pertenece a una ruta válida,
- Respeta la capacidad máxima del vehículo,
- Calcula peso transportado,
- Calcula consumo de combustible,
- Registra fecha de salida y llegada.

Además, los estados fueron distribuidos utilizando probabilidades controladas para simular una operación real.

Entregas

Cada entrega pertenece obligatoriamente a un viaje.

Se generan:

- Cliente,
- Dirección,
- Peso,
- Horario programado,
- Horario real,
- Firma del receptor,
- Estado de la entrega.

La cantidad de entregas por viaje varía entre dos y seis paquetes.

Mantenimientos

Los mantenimientos se generan proporcionalmente al uso de cada vehículo.

Para ello se analizó:

- Cantidad de viajes realizados,
- Periodo de operación,
- Tipo de vehículo.

Esto evita asignar mantenimientos irreales a vehículos poco utilizados.

2.5 Control de calidad de datos

Después de generar la información se ejecutó una etapa completa de validación.

Entre las verificaciones realizadas se encuentran:

- Integridad referencial.
- Consistencia temporal.
- Capacidad máxima de carga.
- Consumo de combustible.
- Costos de mantenimiento.
- Distribución estadística de estados.
- Vehículos sin viajes.
- Conductores sin viajes.
- Mantenimientos antes de la compra del vehículo.
- Viajes antes de la contratación del conductor.
- Distribución de entregas.
- Distribución de combustible.
- Duración de viajes.

Estas validaciones permitieron garantizar que los datos fueran consistentes antes de iniciar el análisis.

---

3. Optimización de consultas en PostgreSQL

3.1 Optimización mediante índices

Con el objetivo de mejorar el rendimiento de las consultas más costosas del sistema, se implementaron varios índices sobre las tablas con mayor volumen de datos y sobre las columnas utilizadas con mayor frecuencia en filtros (WHERE), uniones (JOIN) y ordenamientos (ORDER BY).

Los índices creados fueron los siguientes:

Índice	Tabla	Objetivo
idx_trips_composite_joins	trips	Optimizar los JOIN entre viajes, vehículos, conductores y rutas para consultas sobre viajes completados.
idx_trips_departure	trips	Mejorar las búsquedas por rangos de fechas utilizando departure_datetime.
idx_deliveries_scheduled_datetime	deliveries	Optimizar consultas que filtran entregas realizadas por fecha programada.
idx_deliveries_status	deliveries	Acelerar consultas que filtran por estado de la entrega.
idx_maintenance_vehicle_cost	maintenance	Optimizar la búsqueda de mantenimientos asociados a cada vehículo.
idx_drivers_status_license	drivers	Mejorar las consultas sobre conductores activos y vencimiento de licencias.
idx_routes_metrics	routes	Facilitar el acceso a información de rutas utilizada en cálculos de distancia y destino.
idx_vehicles_status	vehicles	Optimizar filtros sobre el estado de los vehículos.

Es importante mencionar que la creación de un índice no implica que PostgreSQL lo utilice en todas las consultas. El optimizador del motor de base de datos evalúa diferentes estrategias de ejecución y selecciona aquella cuyo costo estimado sea menor.

En los casos donde el plan de ejecución muestra un Sequential Scan (Seq Scan), esto generalmente ocurre por una de las siguientes razones:

- La tabla contiene pocos registros y recorrerla completamente resulta más económico que acceder mediante un índice.
- La consulta necesita procesar una gran parte de la tabla, por lo que el índice aporta poco beneficio.
- La condición del filtro utiliza funciones o expresiones que impiden aprovechar directamente el índice.
- El mayor costo de la consulta se encuentra en operaciones posteriores como JOIN, GROUP BY, ORDER BY o

funciones de ventana, y no en la búsqueda inicial de registros.

A continuación, se presenta el análisis individual de cada consulta utilizando los resultados obtenidos con EXPLAIN ANALYZE.

3.2 Análisis individual de las consultas

Query 1: Contar vehículos por tipo

Problema de negocio: Conocer la composición de la flota

Consulta 1 – Cantidad de vehículos por tipo

- Tiempo de ejecución: 0.295 ms
- Plan utilizado: Seq Scan + HashAggregate

Aunque existe un índice sobre el estado de los vehículos (idx_vehicles_status), esta consulta no filtra por dicha columna y únicamente necesita recorrer los 200 registros de la tabla para agruparlos por tipo. Debido al tamaño reducido de la tabla, PostgreSQL determina que un recorrido secuencial es la alternativa más eficiente.

Query 2: Listar conductores con licencia próxima a vencer

Problema de negocio: Prevenir problemas legales por licencias vencidas

Consulta 2 – Licencias próximas a vencer

- Tiempo de ejecución: 0.344 ms
- Plan utilizado: Seq Scan

Se creó el índice idx_drivers_status_license para optimizar búsquedas sobre conductores activos y fechas de vencimiento de licencia. Sin embargo, esta consulta únicamente filtra por license_expiry y no por status, condición necesaria para aprovechar el índice parcial. Además, la tabla contiene únicamente 400 registros, por lo que PostgreSQL opta por un recorrido secuencial.

Query 3: Total de viajes por estado

Problema de negocio: Monitorear operaciones en curso

Consulta 3 – Cantidad de viajes por estado

- Tiempo de ejecución: 52.154 ms
- Plan utilizado: Seq Scan + HashAggregate

Aunque existen índices sobre la tabla trips, la consulta necesita leer los 100.000 registros para agruparlos por estado. Como prácticamente toda la tabla debe procesarse, un índice no representa una mejora significativa y PostgreSQL selecciona un recorrido secuencial.

Query 4: Total de entregas por ciudad destino en los últimos 2 meses

Problema de negocio: Identificar demanda por ciudad para planificación de recursos

Consulta 4 – Ciudades con mayor número de entregas

- Tiempo de ejecución: 181.751 ms
- Plan utilizado: Bitmap Index Scan

En esta consulta PostgreSQL aprovecha el índice idx_trips_departure, utilizando un Bitmap Index Scan para recuperar únicamente los viajes correspondientes a los últimos 60 días. Posteriormente realiza los JOIN con las tablas deliveries y routes. Aunque también existen índices sobre rutas y viajes, el optimizador determina que el índice sobre la fecha es el que ofrece la mayor reducción de registros.

Query 5: Conductores activos con cantidad de viajes completados

Problema de negocio: Evaluar carga de trabajo por conductor

Consulta 5 – Rendimiento de conductores activos

- Tiempo de ejecución: 160.908 ms
- Plan utilizado: Seq Scan

Aunque existe el índice idx_drivers_status_license, la tabla drivers es pequeña y la consulta debe recorrer completamente la tabla trips para calcular las métricas de desempeño. El costo principal se encuentra en las agregaciones sobre los viajes y no en la búsqueda de conductores, por lo que PostgreSQL no considera necesario utilizar el índice.

Query 6: Promedio de entregas por conductor en los últimos 6 meses

Problema de negocio: Medir productividad individual de conductores

Consulta 6 – Productividad por conductor

- Tiempo de ejecución: 213.548 ms
- Plan utilizado: Bitmap Index Scan

El índice idx_trips_departure es utilizado para recuperar únicamente los viajes realizados durante los últimos seis meses. Esta reducción inicial permite disminuir el volumen de información antes de ejecutar las uniones y agrupaciones necesarias para calcular la productividad de cada conductor.

Query 7: Rutas con mayor consumo de combustible por kilómetro

Problema de negocio: Identificar rutas ineficientes para optimización

Consulta 7 – Consumo promedio de combustible por ruta

- Tiempo de ejecución: 169.996 ms
- Plan utilizado: Parallel Seq Scan

Aunque se creó el índice idx_routes_metrics, la mayor carga de trabajo corresponde al procesamiento de la tabla trips, donde una gran cantidad de registros cumplen las condiciones del filtro. En este escenario PostgreSQL obtiene un mejor rendimiento distribuyendo el recorrido secuencial entre varios procesos (Parallel Seq Scan) que utilizando un índice.

Query 8: Análisis de entregas retrasadas por día de la semana

Problema de negocio: Identificar patrones de retraso para mejorar planificación

Consulta 8 – Entregas por día de la semana

- Tiempo de ejecución: 64.293 ms
- Plan utilizado: Bitmap Index Scan

La consulta aprovecha el índice idx_deliveries_scheduled_datetime, recuperando unicamente las entregas realizadas durante los últimos 90 días antes de efectuar la agrupación por día de la semana.

Query 9: Costo de mantenimiento por kilómetro recorrido

Problema de negocio: Evaluar costo-beneficio de cada tipo de vehículo

Consulta 9 – Costo de mantenimiento por kilómetro

- Tiempo de ejecución: 2939.531 ms
- Plan utilizado: Index Scan

Esta consulta utiliza el índice idx_maintenance_vehicle_cost para acceder eficientemente a los registros de mantenimiento de cada vehículo. No obstante, sigue siendo la consulta más costosa debido al elevado número de JOIN, agregaciones y cálculos requeridos, además del gran volumen de datos procesados.

Query 10: Ranking de conductores por eficiencia usando Window Functions

Problema de negocio: Identificar top performers para incentivos

Consulta 10 – Ranking de conductores

- Tiempo de ejecución: 230.521 ms
- Plan utilizado: Bitmap Index Scan

Se utiliza el índice idx_trips_departure para limitar el análisis a los viajes de los últimos tres meses. El mayor tiempo de ejecución proviene posteriormente de las funciones de ventana (Window Functions) empleadas para calcular el ranking de los conductores.

Query 11: Análisis de tendencia de viajes con LAG y LEAD

Problema de negocio: Proyectar necesidades futuras basadas en tendencias

Consulta 11 – Tendencia mensual de viajes completados

- Tiempo de ejecución: 146.630 ms
- Plan utilizado: Seq Scan

Aunque existe el índice compuesto idx_trips_composite_joins, este está diseñado principalmente para optimizar uniones entre tablas y no consultas que agrupan únicamente por fecha. Además, aproximadamente el 90 % de los viajes corresponden al estado completed, por lo que PostgreSQL considera más eficiente recorrer toda la tabla que utilizar un índice con baja selectividad.

Query 12: Pivot de entregas por hora y día de la semana

Problema de negocio: Optimizar horarios de operación y personal

Consulta 12 – Distribución de entregas por hora

- Tiempo de ejecución: 266.584 ms
- Plan utilizado: Parallel Seq Scan

Aunque existe el índice idx_deliveries_scheduled_datetime, la consulta aplica la función EXTRACT(HOUR FROM scheduled_datetime). Al utilizar una función sobre la columna indexada, PostgreSQL no puede aprovechar directamente el índice para esa parte del filtro y decide realizar un recorrido secuencial paralelo, que resulta más eficiente para este tipo de procesamiento.

3.3 Comparación de tiempos sin índices y con índices

Consulta	Tiempo sin índices (ms)	Tiempo con índices (ms)	Diferencia (ms)	Mejora (%)	Observación
1	0.191	0.295	+0.104	-54.5%	Sin cambios. La tabla tiene solo 200 registros, el optimizador mantiene Seq Scan.
2	0.141	0.344	+0.203	-144.0%	También es una tabla pequeña; el índice no aporta beneficio.
3	35.274	52.154	+16.880	-47.9%	Consulta agregada sobre toda la tabla; debe leer todos los registros.
4	213.325	181.751	-31.574	14.8%	El índice sobre fecha reduce el conjunto de viajes antes del JOIN.
5	69.153	160.908	+91.755	-132.7%	El índice no ayuda porque la consulta sigue leyendo toda la tabla Trips.
6	153.992	213.548	+59.556	-38.7%	El filtro usa índice, pero el costo de JOIN y GROUP BY domina el tiempo.
7	188.014	169.996	-18.018	9.6%	Ligera mejora gracias al índice sobre fechas.
8	145.466	64.293	-81.173	55.8%	Gran mejora. Se reemplaza Seq Scan por Bitmap Index Scan sobre scheduled_datetime.
9	12130.860	2939.531	-9191.329	75.8%	Mayor mejora del proyecto. El índice elimina millones de comparaciones y usa Index Scan + Memoize.
10	163.943	230.521	+66.578	-40.6%	El índice filtra pocos registros; el costo principal sigue siendo la agregación.
11	148.604	146.630	-1.974	1.3%	Diferencia mínima; la ordenación sigue siendo el cuello de botella.
12	262.057	266.584	+4.527	-1.7%	Sin cambios significativos. Continúa usando Parallel Seq Scan.

La comparación demuestra que los índices son más efectivos cuando la consulta realiza búsquedas selectivas sobre

columnas utilizadas en WHERE, JOIN o ORDER BY. Cuando una consulta necesita procesar la mayor parte de la tabla o ejecutar agregaciones complejas, el optimizador suele preferir un Sequential Scan, ya que el costo adicional de acceder al índice supera el beneficio obtenido. Por ello, la creación de índices debe realizarse con base en el patrón de consultas y no de manera indiscriminada.

---

4. Arquitectura AWS

4.1 Introducción

Este documento describe el diseño de la arquitectura AWS implementada para FleetLogix, explicando los servicios utilizados, el proceso de configuración de la infraestructura, el funcionamiento de las funciones Lambda y el procedimiento de migración de la base de datos PostgreSQL hacia Amazon RDS. El objetivo es documentar el comportamiento del sistema sin necesidad de ejecutarlo, permitiendo comprender cómo interactúan los diferentes componentes dentro de la solución propuesta.

4.2 Visión general de la arquitectura

> **[Imagen del PDF original: diagrama “FleetLogix – Arquitectura AWS para Análisis en Tiempo Real” (plataforma serverless e integrada para monitoreo, análisis y toma de decisiones en operaciones logísticas, región us-east-1). Se organiza en: Fuentes de datos (PostgreSQL operacional on-premise, hojas de cálculo y CSV, GPS/telemática, app móvil, APIs externas); Ingesta de datos (AWS DMS, AWS Glue Data Catalog, Amazon Kinesis Data Streams); Almacenamiento (Amazon S3 Data Lake con raw-data, processed-data, curated-data, backups y logs); Procesamiento (AWS Glue ETL Jobs, Amazon EMR con Apache Spark, AWS Lambda, Amazon Athena); Análisis y almacén (Amazon Redshift como Data Warehouse, Redshift Spectrum, Amazon QuickSight); Visualización y usuarios (dashboards, usuarios de negocio, analistas de datos, app móvil); Amazon DynamoDB para estado en tiempo real (deliveries_status, vehicle_tracking, routes_waypoints, alerts_history); Amazon RDS for PostgreSQL (fleetlogix-db) con AWS Backup; funciones Lambda (lambda_verificar_entrega con API Gateway, lambda_calcular_eta con EventBridge cada 5 minutos, lambda_alerta_desvio con Kinesis Data Streams) y Amazon SNS para alertas; servicios transversales (IAM, CloudWatch, SNS, CloudTrail, KMS y S3 para logs y archivos). Beneficios: escalabilidad automática, alta disponibilidad y resiliencia, procesamiento en tiempo real, optimización de costos, seguridad integrada y análisis avanzado.]**

La arquitectura de FleetLogix fue diseñada para integrar el procesamiento de datos históricos y el monitoreo en tiempo real dentro de un entorno escalable y seguro en AWS. El objetivo principal es centralizar la información proveniente de diferentes fuentes, transformarla mediante procesos ETL y ponerla a disposición de los usuarios de negocio a través de dashboards e indicadores que faciliten la toma de decisiones.

El flujo de datos inicia con las fuentes operacionales, donde PostgreSQL almacena la información transaccional generada por la operación logística, incluyendo vehículos, conductores, rutas, viajes, entregas y mantenimientos. Adicionalmente, la plataforma puede recibir información desde archivos CSV, hojas de cálculo, dispositivos GPS instalados en los vehículos, aplicaciones móviles utilizadas por los conductores y APIs externas que proporcionan información como tráfico, clima y mapas.

Para la ingesta de datos, AWS Database Migration Service (DMS) replica continuamente la información almacenada en PostgreSQL hacia la nube, permitiendo mantener sincronizados los datos sin afectar el rendimiento del sistema transaccional. Paralelamente, AWS Glue Data Catalog registra la estructura y el esquema de los datos, facilitando su descubrimiento y utilización por parte de los diferentes servicios analíticos. Cuando se generan eventos en tiempo real, como actualizaciones de ubicación GPS o cambios en el estado de una entrega, Amazon Kinesis recibe estos flujos de información para su procesamiento inmediato.

Toda la información es almacenada inicialmente en Amazon S3, el cual funciona como el Data Lake de la solución. Dentro de este repositorio se organizan diferentes zonas según el estado de los datos: una zona Raw donde se conservan los datos originales, una zona Processed con información previamente transformada y validada, una zona Curated con datos optimizados para análisis, además de áreas destinadas al almacenamiento de respaldos y registros de ejecución del sistema.

El procesamiento de los datos es realizado principalmente por AWS Glue, donde se ejecutan los procesos ETL encargados de extraer la información, limpiarla, validar reglas de negocio, calcular indicadores y preparar los datos para su análisis. Cuando el volumen de información aumenta considerablemente, Amazon EMR utiliza Apache Spark para distribuir las cargas de trabajo y procesar grandes cantidades de registros de forma paralela. Para consultas rápidas directamente

sobre los archivos almacenados en S3, Amazon Athena permite ejecutar consultas SQL sin necesidad de cargar previamente los datos en una base de datos.

Una vez transformada, la información es cargada en Amazon Redshift, que actúa como el Data Warehouse de la organización. Allí se implementa un modelo dimensional compuesto por tablas de dimensiones y tablas de hechos, optimizado para consultas analíticas de alto rendimiento. En aquellos casos donde se requiere consultar directamente archivos almacenados en el Data Lake, Redshift Spectrum permite acceder a la información sin necesidad de duplicarla dentro del Data Warehouse.

Los datos consolidados son consumidos por Amazon QuickSight, donde se construyen dashboards interactivos e indicadores de desempeño relacionados con entregas, consumo de combustible, eficiencia operativa, desempeño de conductores, tiempos de entrega y demás métricas estratégicas. Estos tableros son utilizados tanto por usuarios de negocio como por analistas de datos para apoyar la toma de decisiones basadas en información actualizada.

Además del procesamiento histórico, la arquitectura incorpora un componente de procesamiento en tiempo real. La información proveniente de dispositivos GPS y aplicaciones móviles es almacenada temporalmente en Amazon DynamoDB, donde se mantiene el estado actual de las entregas, la ubicación de los vehículos, los puntos de las rutas y el historial de alertas generadas durante la operación.

Sobre esta información actúan varias funciones implementadas con AWS Lambda. Estas funciones ejecutan procesos automáticos como la verificación del estado de una entrega, el cálculo periódico del tiempo estimado de llegada (ETA) y la detección de desvíos de ruta mediante el análisis continuo de la información recibida desde Amazon Kinesis. Cuando se identifica un evento importante, las funciones generan notificaciones utilizando Amazon SNS, permitiendo alertar oportunamente a los equipos de operación.

Toda la plataforma se encuentra respaldada por servicios transversales que garantizan la seguridad, disponibilidad y monitoreo de la solución. AWS IAM administra la autenticación y los permisos de acceso a los diferentes recursos, AWS CloudWatch supervisa el rendimiento del sistema y registra métricas y eventos, AWS CloudTrail mantiene un historial de auditoría de todas las acciones realizadas dentro de la infraestructura, AWS KMS protege la información mediante cifrado y AWS Backup automatiza la generación de copias de seguridad de los servicios críticos.

Gracias a esta arquitectura, FleetLogix dispone de una plataforma moderna, escalable y altamente disponible que integra el procesamiento de datos históricos con análisis en tiempo real, permitiendo optimizar la operación logística, automatizar procesos críticos y ofrecer información confiable para la toma de decisiones estratégicas.

4.3 Recursos creados en aws_setup.py

El archivo aws_setup.py automatiza la creación y configuración de la infraestructura básica de AWS necesaria para soportar la plataforma FleetLogix. Su propósito es preparar el entorno donde posteriormente se ejecutarán los procesos ETL, las funciones Lambda y el almacenamiento de información histórica y en tiempo real.

Amazon RDS PostgreSQL

El script crea una instancia administrada de PostgreSQL utilizando Amazon RDS.

La configuración establece una instancia db.t3.micro, correspondiente al nivel gratuito de AWS, con PostgreSQL versión 15.4, 20 GB de almacenamiento, respaldos automáticos durante siete días y ventanas definidas para mantenimiento y copias de seguridad.

El propósito de esta instancia es alojar la base de datos operacional de FleetLogix dentro de la nube, eliminando la necesidad de administrar un servidor propio y garantizando alta disponibilidad y recuperación ante fallos.

Amazon S3

Posteriormente se crea un bucket denominado fleetlogix-data, utilizado como Data Lake del proyecto.

Dentro del bucket se crean automáticamente las carpetas:

- raw-data/
- processed-data/
- backups/
- logs/

Además, se configura una regla de ciclo de vida (Lifecycle Policy) que mueve automáticamente los datos almacenados en

la carpeta raw-data hacia Glacier después de 90 días, reduciendo significativamente los costos de almacenamiento.

Amazon DynamoDB

El script crea cuatro tablas NoSQL destinadas al procesamiento en tiempo real.

deliveries_status

Almacena el estado actual de cada entrega, permitiendo conocer si una orden se encuentra pendiente, en tránsito o entregada.

vehicle_tracking

Registra continuamente la ubicación GPS de cada vehículo junto con la fecha y hora de la actualización.

routes_waypoints

Almacena los puntos GPS que conforman cada ruta planificada, información utilizada posteriormente para detectar desviaciones.

alerts_history

Conserva el historial completo de alertas generadas por el sistema durante la operación logística.

AWS Backup

Después de crear la instancia RDS, el script genera un snapshot inicial y configura una política de retención de siete días para los respaldos automáticos.

Esto garantiza la recuperación de la información ante posibles fallos del sistema.

AWS IAM

Finalmente se crea un rol denominado FleetLogixLambdaRole, el cual será utilizado por todas las funciones Lambda.

A este rol se le asignan las políticas necesarias para acceder a:

- AWS Lambda Basic Execution
- Amazon DynamoDB
- Amazon S3
- Amazon SNS

De esta manera las funciones Lambda pueden consultar tablas, almacenar información y enviar notificaciones sin utilizar credenciales explícitas.

4.4 Funciones implementadas en lambda_functions.py

El archivo lambda_functions.py contiene tres funciones Lambda diseñadas para automatizar procesos críticos de la operación logística mediante un modelo completamente serverless.

Cada función responde a un evento diferente y ejecuta tareas específicas dentro de la plataforma.

Función Lambda: Verificar Entrega

La función lambda_verificar_entrega() tiene como objetivo verificar si una entrega ya fue completada consultando directamente la tabla deliveries_status almacenada en Amazon DynamoDB.

Cuando la aplicación móvil registra una entrega, envía un evento que contiene el identificador de la entrega (delivery_id) y, opcionalmente, el número de seguimiento (tracking_number).

La función valida inicialmente que el identificador haya sido recibido. Si este dato no existe, responde inmediatamente con un código HTTP 400 indicando que el parámetro es obligatorio.

Posteriormente establece conexión con la tabla deliveries_status y busca el registro correspondiente mediante la clave primaria delivery_id.

Si el registro existe, recupera el estado actual de la entrega, verifica si dicho estado corresponde a delivered y construye

una respuesta que incluye el identificador de la entrega, el número de seguimiento, el estado actual y la fecha de entrega registrada.

Cuando la entrega no existe en DynamoDB, la función devuelve un código HTTP 404 indicando que el recurso solicitado no fue encontrado.

En caso de producirse cualquier excepción durante la consulta, la función captura el error y devuelve un código HTTP 500 junto con el mensaje correspondiente.

Este proceso permite que la aplicación móvil consulte en tiempo real el estado actualizado de cualquier entrega sin necesidad de acceder directamente a la base de datos principal.

Función Lambda: Calcular ETA

La función lambda_calcular_eta() calcula el tiempo estimado de llegada (ETA) de un vehículo utilizando la ubicación actual, el destino y la velocidad reportada.

Para ello recibe como parámetros el identificador del vehículo, la ubicación GPS actual, las coordenadas del destino y la velocidad promedio.

La distancia restante se calcula mediante una aproximación simplificada basada en coordenadas geográficas. Posteriormente se estima el tiempo restante utilizando la velocidad del vehículo.

Finalmente, la información calculada se almacena en la tabla vehicle_tracking, permitiendo que aplicaciones móviles y dashboards consulten permanentemente el progreso de cada vehículo.

Función Lambda: Alerta de Desvío

La función lambda_alerta_desvio() supervisa continuamente la ubicación GPS de los vehículos.

Cada vez que recibe una nueva posición consulta en DynamoDB los waypoints correspondientes a la ruta asignada.

Posteriormente calcula la distancia mínima entre la posición actual del vehículo y la ruta planificada.

Cuando dicha distancia supera el umbral establecido de cinco kilómetros, el sistema considera que existe un desvío de ruta.

En ese momento se genera un mensaje con la información del incidente, el cual es enviado mediante Amazon SNS para notificar al equipo de operaciones.

Adicionalmente la alerta queda registrada en la tabla alerts_history, permitiendo conservar un historial completo de todos los eventos detectados.

4.5 Explicación de la función migrar_datos_postgresql()

La función migrar_datos_postgresql() no realiza directamente la migración de la base de datos. Su propósito consiste en generar automáticamente un script Bash denominado migrate_to_rds.sh, el cual contiene todas las instrucciones necesarias para copiar la información desde PostgreSQL local hacia Amazon RDS.

El proceso comienza realizando una copia completa de la base de datos local mediante el comando pg_dump, generando un archivo SQL que contiene tanto la estructura como los datos de todas las tablas.

Posteriormente se establece conexión con la instancia PostgreSQL alojada en Amazon RDS y se crea la base de datos destino.

Finalmente, el archivo generado por pg_dump es restaurado utilizando psql, recreando la estructura original e importando toda la información dentro de la instancia administrada por AWS.

Este procedimiento permite migrar completamente la base de datos operacional hacia la nube sin modificar la estructura existente, facilitando la continuidad de los procesos ETL y del entorno analítico implementado en FleetLogix.

---

5. Construcción del Data Warehouse y proceso ETL

Con el fin de soportar procesos analíticos y facilitar la toma de decisiones basada en datos, se diseñó e implementó un

proceso ETL (Extract, Transform, Load) que permite integrar la información operacional almacenada en PostgreSQL dentro de un Data Warehouse implementado en Snowflake.

El proceso automatiza la extracción de los datos, aplica reglas de transformación y calidad, y carga la información utilizando un modelo dimensional optimizado para consultas analíticas.

Como punto de partida, se proporcionó un código base para la implementación del Data Warehouse en Snowflake que presentaba funcionalidades incompletas y diversos errores que afectaban la carga, transformación y organización de los datos. Por esta razón, fue necesario realizar un proceso de análisis, depuración y desarrollo para completar las secciones faltantes, corregir las inconsistencias detectadas y garantizar que la información fuera cargada correctamente, preservando la integridad de los datos y el funcionamiento adecuado del modelo analítico.

5.1 Arquitectura del proceso ETL

El flujo implementado consta de tres etapas principales:

Extracción (Extract)

Se obtiene la información desde la base de datos PostgreSQL a partir de las tablas operacionales:

- Vehicles
- Drivers
- Routes
- Trips
- Deliveries
- Maintenance

Transformación (Transform)

Durante esta etapa se realizaron procesos de limpieza, validación y enriquecimiento de los datos.

Entre las transformaciones implementadas se encuentran:

- cálculo del tiempo real de entrega;
- duración de cada viaje;
- eficiencia de combustible;
- indicadores de puntualidad;
- costos por entrega;
- generación de dimensiones de fecha y hora;
- validación de pesos y tiempos;
- eliminación de registros inconsistentes.

Estas reglas permiten garantizar que únicamente información válida sea cargada al Data Warehouse.

Carga (Load)

Finalmente, la información es cargada en Snowflake mediante procesos automáticos utilizando tablas de staging y operaciones MERGE para mantener actualizadas las dimensiones y la tabla de hechos.

5.2 Modelo dimensional

Se diseñó un esquema estrella (Star Schema) compuesto por una tabla de hechos y múltiples dimensiones.

Tabla de hechos

- FACT_DELIVERIES

Contiene las principales métricas del negocio, incluyendo:

- tiempos de entrega;
- peso transportado;
- consumo de combustible;
- costos;

- indicadores de cumplimiento.

Tablas de dimensiones

- DIM_CUSTOMERS
- DIM_VEHICLES
- DIM_DRIVERS
- DIM_ROUTES
- DIM_DATE
- DIM_TIME

Cada dimensión utiliza claves sustitutas (Surrogate Keys) para facilitar el análisis histórico y mejorar el rendimiento de las consultas.

5.3 Calidad de datos durante el ETL

Antes de realizar la carga al Data Warehouse se aplicaron múltiples validaciones, entre ellas:

- eliminación de tiempos negativos;
- validación de pesos de paquetes;
- consistencia de fechas;
- normalización de tipos de datos;
- control de valores nulos;
- validación de reglas de negocio.

Este proceso permitió garantizar la consistencia de la información analítica.

5.4 Automatización del proceso

El proceso ETL fue desarrollado completamente en Python y puede ejecutarse de forma automática.

Durante cada ejecución se registran:

- cantidad de registros extraídos;
- cantidad de registros transformados;
- cantidad de registros cargados;
- tiempo total de ejecución;
- errores encontrados durante el proceso.

Esto facilita el monitoreo y la trazabilidad de la carga de datos.

5.5 Validación del Data Warehouse

Una vez finalizada la carga se verificó:

- consistencia de claves sustitutas;
- correspondencia entre dimensiones y tabla de hechos;
- integridad de las relaciones;
- consistencia de métricas calculadas;
- correcto funcionamiento del proceso de carga.

Las pruebas confirmaron que la información almacenada en el Data Warehouse es consistente con los datos operacionales del sistema.